

Threshold Signatures

Term Project Report

Team 5

22CS30043 Raaja Das

22CS30056 Sumit Kumar

22CS30067 Alla Sarmada

CS60088 Foundations of Cryptography

April 15, 2026

Abstract

Digital signatures provide authenticity, but they concentrate trust: a single private key held by a single party is a single point of failure. Threshold signature schemes distribute this trust across n parties so that any k of them can jointly produce a valid signature, while fewer than k gain nothing useful even if they cooperate. This report surveys the topic of threshold signatures through three research papers that together cover the foundational constructions, their security proofs, and practical extensions.

The first paper we study is Shoup’s “Practical Threshold Signatures” (Eurocrypt 2000), which presents the first RSA-based threshold signature scheme that is simultaneously non-interactive, has compact shares, and comes with a rigorous security proof in the random oracle model. The second paper, Boldyreva’s “Threshold Signatures, Multisignatures and Blind Signatures Based on the Gap-Diffie-Hellman-Group Signature Scheme” (PKC 2003), constructs a threshold scheme over GDH groups that avoids both the trusted dealer and the zero-knowledge proofs required by Shoup, at the cost of requiring pairing-friendly groups and a stricter $t < n/2$ corruption bound. The third paper, Komlo and Goldberg’s “FROST: Flexible Round-Optimized Schnorr Threshold Signatures” (2020), addresses round efficiency by reducing threshold Schnorr signing to two rounds (or one with preprocessing), standardised as RFC 9591 and deployed in real-world systems such as Zcash.

Together, these papers trace a line from basic constructions to more practical and expressive schemes, and show how the standard toolkit of Shamir secret sharing, Lagrange interpolation, and zero-knowledge proofs can be assembled to achieve a property that initially seems hard: splitting signing authority without any single party ever holding the full key.

Contents

1	Introduction	4
1.1	Shamir Secret Sharing and Its Role in Threshold Signatures	4
2	Practical Threshold Signatures (Shoup, 2000)	7
2.1	Background and What the Paper Does	7
2.2	The Security Model	7
2.3	Protocol 1: The Basic Construction	8
2.3.1	Dealer Setup	8
2.3.2	The Δ Trick and Lagrange Interpolation	9
2.3.3	Generating and Verifying Signature Shares	9
2.3.4	Combining Shares into a Signature	10
2.4	Protocol 2: Extending to $k \geq t + 1$	10

2.5	Security Analysis	11
2.5.1	Theorem 1: Security of Protocol 1 for $k = t + 1$	11
2.5.2	Theorem 2: Security of Protocol 2 for $k \geq t + 1$	12
2.6	Observations and Critique	13
3	Threshold Signatures from Gap Diffie-Hellman Groups (Boldyreva, 2003)	15
3.1	What This Paper Does and Why It Matters	15
3.2	Background: GDH Groups and the BLS Signature	15
3.2.1	Gap Diffie-Hellman Groups	15
3.2.2	The BLS Signature Scheme	16
3.3	The Threshold GDH Signature Scheme TGS	16
3.3.1	Security Model and Definitions	16
3.3.2	Key Generation: No Trusted Dealer	17
3.3.3	Signing Protocol	17
3.4	Security Analysis	18
3.5	Proactive Security	19
3.6	The GDH Multisignature Scheme MGS	19
3.7	The Blind GDH Signature Scheme BGS	20
3.8	Observations and Critique	21
4	FROST: Flexible Round-Optimized Schnorr Threshold Signatures	22
4.1	Introduction and Motivation	22
4.2	Background and Prerequisites	22
4.2.1	Notation	22
4.2.2	Shamir Secret Sharing	22
4.2.3	Additive Secret Sharing and Share Conversion	23
4.2.4	Distributed Key Generation (DKG)	23
4.2.5	Schnorr Signatures	23
4.3	The FROST Threshold Signature Scheme	23
4.3.1	Assumptions	23
4.3.2	Key Generation (Ped-DKG)	23
4.3.3	Two-Round Interactive Signing	24
4.3.4	Single-Round Signing with Preprocessing	24
4.4	Mathematical Proofs of Security	25
4.4.1	Assumptions	25
4.4.2	Correctness Proof	25
4.4.3	Unforgeability (EUF-CMA)	26
4.4.4	Adaptive Security Considerations	27
4.5	Security and Efficiency Analysis	27

4.5.1	Security Properties	27
4.5.2	Efficiency Comparison	27
4.5.3	Drawbacks and Limitations	28
4.6	Overview of ROAST	28
4.7	Conclusion and Future Research Directions	29

1 Introduction

Most cryptographic protocols are designed around a single signer. A certificate authority has one private key; a server authenticating to a client uses one key. This simplicity is convenient, but it creates a serious problem in settings where the signer is a high-value target. If that one key is stolen, the attacker can impersonate the signer forever. If the machine holding the key goes down, signing stops entirely. For a root CA or a financial institution's signing infrastructure, neither outcome is acceptable.

Threshold signature schemes address this by distributing the signing key across n parties in such a way that any k of them can cooperate to produce a valid signature, but any coalition smaller than k cannot sign anything, and in fact learns nothing about the private key. The security guarantee is analogous to Shamir's secret sharing [?]: the key is shared, not replicated. This is fundamentally different from simply storing the same key on multiple machines. Threshold security means that even if up to $k - 1$ machines are fully compromised, the key itself remains secret and the attacker cannot forge signatures.

1.1 Shamir Secret Sharing and Its Role in Threshold Signatures

Shamir's Secret Sharing (SSS) [?] is the foundational building block for most threshold signature schemes. It distributes a secret s among n participants so that any subset of size at least t (the threshold) can reconstruct s , while any subset of size $t - 1$ learns nothing about s . The scheme is information-theoretically secure and works over a finite field $\mathbb{F}_q$ with $q > n$.

Share generation. A dealer chooses a random polynomial $f(x) \in \mathbb{F}_q[x]$ of degree at most $t - 1$:

$$f(x) = a_0 + a_1x + \cdots + a_{t-1}x^{t-1},$$

with constant term $a_0 = s$. Coefficients $a_1, \dots, a_{t-1}$ are uniform in $\mathbb{F}_q$. For each participant $i = 1, \dots, n$, the dealer assigns a distinct non-zero evaluation point $x_i \in \mathbb{F}_q^\times$ (public) and computes the share $y_i = f(x_i)$.

Reconstruction via Lagrange interpolation. Given any t distinct shares (x_{i_j}, y_{i_j}) , the polynomial f is uniquely determined. The secret is $s = f(0) = \sum_{j=1}^t y_{i_j} \cdot \ell_j(0)$, where

$$\ell_j(x) = \prod_{\substack{k=1 \\ k \neq j}}^t \frac{x - x_{i_k}}{x_{i_j} - x_{i_k}}.$$

Proof of correctness. Any t distinct shares uniquely determine s . A polynomial of degree $\leq t - 1$ over a field is uniquely determined by its values at t distinct points. Since

f has degree $\leq t - 1$ and passes through the given t points, $f(0) = s$ is fixed.

Proof of security (information-theoretic). *Any set of at most $t - 1$ shares reveals no information about s .* Fix $t - 1$ shares (x_i, y_i) . For any candidate secret $s' \in \mathbb{F}_q$, there exists exactly one polynomial f' of degree $\leq t - 1$ satisfying $f'(0) = s'$ and $f'(x_i) = y_i$ (interpolate through the t points $(0, s'), (x_1, y_1), \dots$). Because the dealer chose the coefficients $a_1, \dots, a_{t-1}$ uniformly, every s' is equally likely given the $t - 1$ shares. Hence the conditional distribution of s is uniform over $\mathbb{F}_q$; no information is leaked.

From SSS to threshold signatures. In a threshold signature scheme, the private signing key $\mathbf{sk}$ is taken as the secret s and shared using SSS. To sign a message m , any t parties each compute a *partial signature* using their share and a public function (e.g., in Schnorr or ECDSA, the nonce is also shared). Lagrange interpolation is then applied *in the exponent* of the cryptographic group to combine the partial signatures into a full signature. Because $\mathbf{sk}$ is never reconstructed, the scheme remains secure as long as fewer than t parties are corrupted. The information-theoretic security of SSS guarantees that $t - 1$ corrupted parties gain no advantage in forging signatures.

The topic has direct practical relevance. Certificate authorities using threshold signing are more resilient to key theft. Cryptocurrency wallets that use threshold signatures (sometimes called “multi-sig”) protect funds even if some signers are compromised. Distributed systems that need to produce signed messages for consensus, as in Byzantine fault-tolerant protocols, benefit from threshold signatures that can tolerate up to a third of the parties being malicious. In all of these settings, what is needed is a scheme that is efficient, non-interactive if possible, and comes with a proof of security rather than just heuristic arguments.

Constructing such schemes turns out to be harder than it looks. The naive approach of having all k parties compute partial signatures and combine them runs into the problem that the combination needs to reconstruct the private key implicitly, without any party ever computing it explicitly. For RSA-based signatures, this requires doing polynomial interpolation in a group whose order $\phi(n)$ is secret, which creates technical difficulties that derailed a decade of prior work. For discrete-log-based signatures, the inherent randomness in signing forces interaction among the parties to generate shared nonces. Getting around these obstacles requires non-trivial ideas.

This report covers three papers that together give a solid picture of where threshold signature research stands.

The first, Shoup’s “Practical Threshold Signatures” [1], is a landmark result for threshold RSA. The scheme it presents is surprisingly simple, using little more than Shamir secret sharing and a scaling trick with $l!$ to bypass the $\phi(n)$ -interpolation obstacle, yet it is the first to simultaneously achieve non-interactive signing, compact shares, and a

rigorous proof of security under the RSA assumption. We study this paper in detail in Section 2, including both protocols and their security proofs.

The second, Boldyreva’s “Threshold Signatures, Multisignatures and Blind Signatures Based on the Gap-Diffie-Hellman-Group Signature Scheme” [2], takes a different algebraic route by working in groups where the group order is public (GDH groups). This lets Lagrange interpolation work directly over $\mathbb{Z}_p$ without any scaling factor, removes the need for a trusted dealer by using distributed key generation, and eliminates zero-knowledge proofs from robustness checking. The paper also extends the same framework to multisignatures and blind signatures. We cover it in Section 3.

The third, Komlo and Goldberg’s “FROST: Flexible Round-Optimized Schnorr Threshold Signatures” [3], tackles the round complexity problem for threshold Schnorr schemes. Prior work required three or more rounds per signing session due to the need for jointly generated nonces. FROST reduces this to two rounds (or one with a preprocessing phase) by using non-interactive additive secret sharing combined with per-participant binding factors that prevent adaptive attacks on the aggregated nonce. It has been standardised as RFC 9591 and adopted in practice. We cover it in Section 4.

Throughout, we try not just to describe the constructions but to explain the choices made specifically, why certain assumptions are needed, where the proofs are tight and where they are not, and what the gaps between theory and practice look like.

2 Practical Threshold Signatures (Shoup, 2000)

2.1 Background and What the Paper Does

The starting point for this paper is a natural problem: RSA signing in practice usually means one party holds the private key, and if that party is compromised or unavailable, everything breaks. Threshold signatures distribute the signing capability so that no single party holds the full key. Instead, k out of l parties must cooperate to sign, and any fewer than k learn nothing useful even if they collude.

The idea of threshold signatures had been around since Desmedt's work in 1987 [?], and several schemes had been proposed over the years based on both discrete log and RSA. But as Shoup points out, all of them suffered from at least one of three problems: no rigorous security proof, interactive signing that requires a synchronous network, or signature shares whose size grows linearly in the number of players.

The reason discrete-log-based schemes are stuck being interactive is fundamental to how they work. Schnorr, DSA, ElGamal-type signatures all require a fresh random nonce per signature, and generating that nonce jointly among k parties requires interaction. There is no way around it. RSA, on the other hand, is deterministic once you fix the message hash, which makes non-interactive threshold signing plausible in principle.

The problem people kept running into for threshold RSA was polynomial interpolation. The natural approach is to secret-share the private key d using Shamir's scheme, where players hold shares $s_i = f(i) \bmod \phi(n)$ of a polynomial f with $f(0) = d$, and combine using Lagrange interpolation in the exponent. But players don't know $\phi(n)$ (knowing it would let them factor n), so you can't do arithmetic modulo $\phi(n)$. Previous work tried to work around this by changing the ring they interpolate over, which led to complicated schemes with large shares [?, ?].

Shoup's insight is that the interpolation-over- $\mathbb{Z}_{\phi(n)}$ problem is essentially a non-issue. You don't interpolate modulo $\phi(n)$; you do the interpolation over the integers, scaling by $\Delta = l!$ to clear denominators. The Δ factor then gets absorbed into the exponents during share combination, and everything works out. The scheme ends up being strikingly simple, simple enough that Shoup himself remarks it is "truly amazing" that it had not been proposed and analyzed before.

2.2 The Security Model

The setup has l players, a trusted dealer, and an adversary. Two parameters control the scheme: t is the maximum number of players the adversary can corrupt, and k is the threshold (the minimum number of shares needed to produce a valid signature). The constraints $k \geq t+1$ and $l-t \geq k$ are required, ensuring that the adversary never controls enough shares to sign on its own, and that the honest players always have enough shares

to sign regardless.

The adversary is *static*: it decides which t players to corrupt before the protocol begins. Once corrupted, those players' secret key shares are fully known to the adversary. During signing, the adversary can ask any uncorrupted player to sign any message, and it sees the resulting signature shares.

The scheme is **non-forgable** if the adversary cannot produce a valid signature on a message M unless at least $k - t$ uncorrupted players were asked to sign M . When $k = t + 1$, this reduces to: no forged signature on any message that no honest player ever signed.

The scheme is **robust** if a corrupted player cannot prevent signing from succeeding by submitting a bad share. The way robustness is achieved here is through share verification: each share comes with a non-interactive proof of correctness, so invalid shares are detected and discarded before combination.

A threshold signature scheme in this model consists of three algorithms beyond the standard key generation: a share verification algorithm (checks that player i 's share on message M is correctly formed), a signature verification algorithm (same as ordinary RSA), and a share combining algorithm (takes k verified shares and outputs the final signature).

2.3 Protocol 1: The Basic Construction

2.3.1 Dealer Setup

The dealer chooses safe primes $p = 2p' + 1$ and $q = 2q' + 1$, where p' and q' are also prime. Let $n = pq$ and $m = p'q'$. The RSA public exponent e is chosen as a prime greater than l (this constraint on e is important and will matter in the combination step). The private key is d satisfying $de \equiv 1 \pmod{m}$.

The reason for safe primes is that they give $\mathbb{Z}_n^*$ a clean structure. By the Chinese Remainder Theorem, $\mathbb{Z}_n^* \cong \mathbb{Z}_{2p'} \times \mathbb{Z}_{2q'} \cong \mathbb{Z}_m \times \mathbb{Z}_2 \times \mathbb{Z}_2$. The subgroup of squares, Q_n , is then cyclic of order exactly $m = p'q'$. Working inside Q_n is desirable because m has no small prime factors, which is needed both for the DDH assumption to hold in Q_n and for Lagrange interpolation to work (the relevant Vandermonde matrices are invertible modulo m).

The dealer secret-shares d using a degree- $(k - 1)$ polynomial over $\mathbb{Z}$:

$$f(X) = d + a_1X + a_2X^2 + \cdots + a_{k-1}X^{k-1},$$

where $a_1, \dots, a_{k-1}$ are chosen uniformly at random from $\{0, \dots, m - 1\}$. Player i 's secret key share is:

$$s_i = f(i) \pmod{m}.$$

Then the dealer picks a random element $v \in Q_n$, which serves as a base for verification keys. It sets $VK = v$ and $VK_i = v^{s_i}$ for each player i . The public key is (n, e) and these verification keys are public; only s_i goes privately to player i .

2.3.2 The Δ Trick and Lagrange Interpolation

Let $\Delta = l!$. For any k -element signing set $S \subseteq \{1, \dots, l\}$ and any $i \in \{0, \dots, l\} \setminus S$, $j \in S$, define:

$$\lambda_{i,j}^S = \Delta \cdot \frac{\prod_{j' \in S \setminus \{j\}} (i - j')}{\prod_{j' \in S \setminus \{j\}} (j - j')}.$$

This is the standard Lagrange coefficient scaled by Δ . The denominator is a product of at most $k - 1$ pairwise differences of integers from $\{1, \dots, l\}$, so it divides $(l - 1)!$, which in turn divides $\Delta = l!$. Hence $\lambda_{i,j}^S$ is always an integer. From the Lagrange interpolation formula, we get:

$$\Delta \cdot f(i) \equiv \sum_{j \in S} \lambda_{i,j}^S \cdot f(j) \pmod{m}. \quad (3)$$

Setting $i = 0$ gives $\Delta \cdot d \equiv \sum_{j \in S} \lambda_{0,j}^S \cdot s_j \pmod{m}$. The key point is that this identity holds over $\mathbb{Z}_m$, not over $\mathbb{Z}_{\phi(n)}$, and the players can compute everything on the right-hand side since they know their own s_j and the public Δ and S .

2.3.3 Generating and Verifying Signature Shares

A valid RSA signature on message M is a value y satisfying $y^e \equiv H(M) \pmod{n}$, where H is a hash function treated as a random oracle mapping messages to $\mathbb{Z}_n^*$. Let $x = H(M)$.

Player i 's signature share is:

$$x_i = x^{2\Delta s_i} \pmod{n}. \quad (4)$$

The exponent has two parts: the Δ is needed for the Lagrange interpolation to work, and the factor of 2 forces x_i into Q_n (since squaring kills the $\mathbb{Z}_2 \times \mathbb{Z}_2$ part of $\mathbb{Z}_n^*$). Being in Q_n is needed to make the proof of correctness sound.

Along with the share, each player broadcasts a proof that x_i is correctly formed. The statement to prove is:

$$\log_v(VK_i) = \log_{\bar{x}}(x_i^2),$$

where $\bar{x} = x^{4\Delta}$. Both sides equal s_i . This is an equality of discrete logarithms in the same group Q_n , and Chaum and Pedersen [?] gave an interactive proof for it. To make it non-interactive, player i applies the Fiat-Shamir heuristic: pick a random $r \in \{0, \dots, 2^{L(n)+2L_1} - 1\}$ (where $L_1 = 128$ is a secondary security parameter), compute:

$$v' = v^r, \quad x' = \bar{x}^r, \quad c = H'(v, \bar{x}, v_i, x_i^2, v', x'), \quad z = s_i c + r,$$

and publish (z, c) as the proof. To verify a share $(x_i, (z, c))$, check that:

$$c = H'(v, \bar{x}, v_i, x_i^2, v^z \cdot v_i^{-c}, \bar{x}^z \cdot x_i^{-2c}).$$

The whole process is non-interactive: player i can compute and publish (x_i, z, c) without talking to anyone else.

2.3.4 Combining Shares into a Signature

Given k valid shares from a set $S = \{i_1, \dots, i_k\}$, verified using the share verification algorithm, compute:

$$w = \prod_{j \in S} x_j^{2\lambda_{0,j}^S} \bmod n.$$

From equation (3), substituting $x_j^2 = x^{4\Delta s_j}$:

$$w^e = x^{e \cdot \sum_{j \in S} 2\lambda_{0,j}^S \cdot \overbrace{s_j}^{\text{(absorbed into exponent)}}} = x^{4\Delta^2} = x^{e'}, \quad \text{where } e' = 4\Delta^2.$$

Since e is a prime greater than l , and $e' = 4(l!)^2$ only has prime factors $\leq l$, we have $\gcd(e, e') = 1$. Run the extended Euclidean algorithm to find integers a, b with $e'a + eb = 1$, then set:

$$y = w^a \cdot x^b \bmod n.$$

Then $y^e = w^{ea} \cdot x^{eb} = x^{e'a} \cdot x^{eb} = x^{e'a+eb} = x = H(M)$. So y is a valid RSA signature, in exactly the standard format.

2.4 Protocol 2: Extending to $k \geq t + 1$

Protocol 1 is only analyzed for $k = t + 1$ in Theorem 1. For the more general case $k \geq t + 1$, Shoup modifies the construction as follows.

Instead of $s_i = f(i) \bmod m$, the dealer computes:

$$s_i = f(i) \cdot \Delta^{-1} \bmod m,$$

where Δ^{-1} exists modulo m since $\gcd(\Delta, m) = 1$ (because all prime factors of $\Delta = l!$ are at most $l < e$, and $m = p'q'$ has only large prime factors). With this change, the share formula simplifies to $x_i = x^{2s_i}$, and the combination formula uses $e' = 4$ instead of $e' = 4\Delta^2$. The factor Δ disappears from the public-facing parts of the protocol entirely.

There is one additional adjustment: to handle Jacobi symbols cleanly, the dealer adds to the verification key an element $u \in \mathbb{Z}_n^*$ with Jacobi symbol $(u|n) = -1$. Before computing $x = H(M)$, the signer checks the Jacobi symbol of $H(M)$: if it is -1 , multiply by u^e to flip it to $+1$. This keeps $x \in J_n$ (elements with Jacobi symbol $+1$), which is

needed for the group arithmetic to be consistent. The final signature is then adjusted accordingly by dividing out u if necessary.

Protocol 2 is actually simpler than Protocol 1 once you understand the role of Δ . The Δ in Protocol 1 was there purely to make the integer interpolation work out, but it creates an annoyance where players are computing with huge exponents. Protocol 2 moves the Δ adjustment into the setup (the key shares) rather than the signing, and the signing itself becomes cleaner.

2.5 Security Analysis

2.5.1 Theorem 1: Security of Protocol 1 for $k = t + 1$

Theorem 2.1 ([1]). *For $k = t + 1$, Protocol 1 is a secure threshold signature scheme (robust and non-forgable) in the random oracle model for H' , assuming that the standard RSA signature scheme is secure.*

Proof Sketch. The proof constructs a simulator that, given access to an RSA signing oracle (which computes e -th roots on demand), can simulate the entire view of the adversary without knowing the secret d .

Simulating corrupted players' shares. The simulator does not know the actual polynomial f . For the $t = k - 1$ corrupted players $i_1, \dots, i_{k-1}$, it picks their shares s_{i_j} uniformly at random from $\{0, \dots, \lfloor n/4 \rfloor - 1\}$. In the real scheme, shares are uniform in $\{0, \dots, m - 1\}$. The difference is that $n/4 - m = (p' + q')/2 + 1/4 = O(\sqrt{n})$, so the statistical distance between the two distributions is $O(n^{-1/2})$, which is negligible. This step is purely statistical: no computational assumption is needed.

Simulating uncorrupted players' shares. Once the corrupted shares s_{i_j} are fixed, because $k = t + 1$, the k points $\{(0, d), (i_1, s_{i_1}), \dots, (i_{k-1}, s_{i_{k-1}})\}$ completely determine the polynomial f . The shares for uncorrupted players are thus determined, but the simulator doesn't know d . It gets around this using the RSA oracle. Given an RSA signature y with $y^e = x$, the simulator can compute the share $x_i = x^{2\Delta s_i}$ for any uncorrupted player i as:

$$x_i = y^{2(\lambda_{i,0}^S + e(\lambda_{i,i_1}^S s_{i_1} + \dots + \lambda_{i,i_{k-1}}^S s_{i_{k-1}}))},$$

where $S = \{0, i_1, \dots, i_{k-1}\}$. This follows from equation (3) and the fact that $y^e = x$ so the simulator can move between x -powers and y -powers. The verification keys $v_i = v^{s_i}$ are computed similarly.

Simulating the proofs of correctness. The simulator generates valid-looking proofs for uncorrupted players without knowing their s_i directly. It chooses random c and z , then programs the random oracle H' to return c at the point $(v, \bar{x}, v_i, x_i^2, v^z v_i^{-c}, \bar{x}^z x_i^{-2c})$. This is possible as long as the random oracle has not been queried at that point before, which

holds except with negligible probability (since c was chosen after z , and the hash input determines them).

Soundness. Suppose a corrupted player submits an incorrect share x_i with a passing proof (z, c) . Since all elements are in Q_n and v generates Q_n , write $\bar{x} = v^\alpha$, $v_i = v^{s_i}$, $x_i^2 = v^\beta$ in the DL representation. If x_i is wrong then $\beta \neq s_i\alpha$, so $\beta - s_i\alpha \not\equiv 0 \pmod{m}$. The verification equation then forces c to satisfy a non-trivial linear equation modulo p' or modulo q' . Since in the random oracle model the challenge $c = H'(\dots)$ is uniformly distributed and independent, this happens only with probability $O(1/p') = O(n^{-1/2})$, which is negligible.

Non-forgeability. The non-forgeability reduction works as follows. Suppose an adversary $\mathcal{A}$ forges a signature on some message M with non-negligible probability ϵ . We build a reduction that breaks RSA with probability close to ϵ . The reduction runs the simulator above, answering all of $\mathcal{A}$'s queries. When $\mathcal{A}$ outputs a forged signature y^* on some fresh message M^* with $y^{*e} = H(M^*)$, the reduction outputs y^* as an e -th root of $H(M^*)$, breaking RSA. The zero-knowledge property of the simulated proofs ensures the adversary's view in the simulation is statistically close to its view in the real attack. $\square$

2.5.2 Theorem 2: Security of Protocol 2 for $k \geq t + 1$

Theorem 2.2 ([1]). *In the random oracle model for both H and H' , Protocol 2 is a secure threshold signature scheme for all $k \geq t + 1$ under the RSA and DDH assumptions. When $k = t + 1$, the DDH assumption is not needed.*

The reason Protocol 1's proof breaks down for $k > t + 1$ is subtle. When $k > t + 1$, the adversary controls t players but needs $k > t + 1$ shares to sign. This means that even the adversary's target message (the one it is trying to forge) may be requested for signing, and some honest players will contribute shares for it. The simulator in the Protocol 1 proof used the RSA oracle specifically when an honest player's share is requested, which works fine when those shares are determined by the corrupted shares plus d . But with $k > t + 1$, there are more honest players than needed to pin the polynomial, and the simulator would need to commit to the honest players' shares before knowing which message will be the forgery target. This creates a circular dependency that breaks the Protocol 1 reduction.

Shoup resolves this with a sequence of three simulators for Protocol 2.

The **first simulator** handles everything the same way as before: corrupted shares are chosen at random, uncorrupted shares are derived using the RSA oracle. But it cannot generate shares for the target message when $k > t + 1$ because it doesn't yet know which message the adversary will forge.

The **second simulator** guesses the target message upfront (one of the random oracle queries), and for that message specifically, it generates the honest players' shares differently: instead of using g^r as a base (where g is the committed group element), it

uses a separate random h . The DDH assumption says that the adversary cannot tell the difference between $(g, g^r, \dots; h, h^r, \dots)$ and $(g, g^r, \dots; h, h^{b_1}, \dots)$ where the h -exponents are independent. So the adversary's view in the second simulator is computationally indistinguishable from the first.

The **third simulator** goes further: for the target message, it simply sets the hash output $\hat{x} = z$ for a random z of its choice, and simulates the honest players' shares as random quadratic residues. If the adversary forges on the target message without querying too many shares from honest players (fewer than $k - t$ of them, as required by the definition), then the adversary has essentially computed an e -th root of a random element z in $\mathbb{Z}_n^*$, which contradicts the RSA assumption.

The DDH assumption is used to justify the move from the first to the second simulator, and is not needed when $k = t + 1$ because the circular dependency doesn't arise in that case.

2.6 Observations and Critique

The scheme is clean and the security proof is tight, but there are a few points worth thinking about.

The most glaring practical limitation is the trusted dealer. The dealer knows both d (the full private key) and all the shares s_i . A compromised dealer breaks everything. This is not an academic concern: in the settings where threshold signatures are actually needed, like distributed key management for certificate authorities or blockchain multisig, a trusted setup is exactly what you want to avoid. Shoup does not address distributed key generation in this paper; it is treated as an orthogonal problem.

The random oracle model is used in two places. For Protocol 1, it is only needed for the hash H' used in the proof of correctness; the message hash H could be replaced by a standard signature assumption. For Protocol 2, both H and H' are treated as random oracles in a more fundamental way (specifically, the simulator needs to control the outputs of H to set up the target message). This is a stronger use of the random oracle model and is one of Shoup's stated motivations for calling Protocol 1 the preferred scheme when $k = t + 1$.

The exponent $\Delta = l!$ in Protocol 1 is worth discussing. For $l = 20$ players, $\Delta = 20! \approx 2.4 \times 10^{18}$, which means the exponent $2\Delta s_i$ in the share generation is an enormous number even before reducing modulo m . In practice, you reduce modulo m when computing, so the group operations are not affected by the size of Δ itself. But it does mean the exponents you work with in key generation and combination are large. Protocol 2 avoids this entirely by folding Δ^{-1} into the key shares at setup time, which is why Protocol 2 is sometimes preferred even in the $k = t + 1$ case.

The requirement that e be prime and greater than l is a departure from how RSA is

usually used. Typically, $e = 65537$ is used for efficiency. If $l < 65537$, this is compatible; but for very large player sets it would force a larger e , which slows down verification. For the threshold applications Shoup had in mind (e.g., Byzantine agreement with tens to hundreds of nodes), this is not an issue.

Overall, this paper is a genuine contribution: it identified that a major obstacle in prior work was actually not an obstacle, and the resulting scheme is simpler and more efficient than everything that came before it. The security proofs are rigorous and the reduction to standard assumptions (RSA, DDH) is clean.

3 Threshold Signatures from Gap Diffie-Hellman Groups (Boldyreva, 2003)

3.1 What This Paper Does and Why It Matters

Shoup’s scheme from the previous chapter solved threshold RSA, but RSA has a specific limitation: because the group order $\phi(n)$ is secret, Lagrange interpolation must be done over the integers with a cumbersome $\Delta = l!$ scaling factor, and achieving robustness requires zero-knowledge proofs. The obvious question is whether you can do better using a group where the order is public.

Boldyreva’s 2003 paper answers this in the context of Gap Diffie-Hellman (GDH) groups, by building a threshold version of the BLS signature scheme [?]. The resulting construction *TGS* is remarkably clean: share generation is just one exponentiation per player, verification is a single DDH check, and combination is a product of Lagrange-weighted powers. No zero-knowledge proofs. No Δ factors. No trusted dealer. And because the group order p is public, Lagrange interpolation works directly over $\mathbb{Z}_p$ without any scaling tricks.

Beyond threshold signatures, the same paper proposes a multisignature scheme *MGS* and a blind signature scheme *BGS*, all based on the same GDH framework. We cover the threshold scheme in detail and discuss the other two constructions at the end.

3.2 Background: GDH Groups and the BLS Signature

3.2.1 Gap Diffie-Hellman Groups

Let G be a multiplicative group of prime order p . Two standard problems are defined in G :

- **CDH:** Given (g, g^a, g^b) for random $g \in G$ and random $a, b \in \mathbb{Z}_p^*$, compute g^{ab} .
- **DDH:** Given four group elements (g, u, v, h) , decide whether $\log_g u = \log_g v = \log_g h$ (i.e., whether (g, u, v, h) is a valid Diffie-Hellman tuple).

Definition 3.1 ([2]). *A prime-order group G is a Gap Diffie-Hellman (GDH) group if there exists an efficient algorithm $\mathcal{V}_{DDH}$ solving DDH in G , but no polynomial-time algorithm solves CDH in G .*

The name comes from the “gap” between these two problems: deciding is easy, computing is hard. The canonical example is an elliptic curve group with a bilinear pairing $e : G \times G \rightarrow G_T$. Bilinearity gives $e(g^a, g^b) = e(g, g)^{ab}$, so to check whether (g, u, v, h) is a DH tuple, it suffices to check $e(g, h) = e(u, v)$. This check is efficient, while computing the actual DH value from (g, g^a, g^b) remains hard.

3.2.2 The BLS Signature Scheme

The GDH signature scheme $GS[G]$ of Boneh, Lynn, and Shacham [?] works as follows. Let $H : \{0, 1\}^* \rightarrow G^*$ be a hash function (random oracle). The global information $I = (p, g, H)$ is public.

- $\mathcal{K}(I)$: Pick $x \leftarrow \mathbb{Z}_p^*$, set $y = g^x$. Public key is (p, g, H, y) , secret key is x .
- $\mathcal{S}(I, sk, M)$: Compute $\sigma = H(M)^x$. Output (M, σ) .
- $\mathcal{V}(pk, M, \sigma)$: Check $\mathcal{V}_{DDH}(g, y, H(M), \sigma) = 1$.

Verification works because a valid signature satisfies $\log_g y = \log_{H(M)} \sigma = x$, so $(g, y, H(M), \sigma)$ is a DH tuple. The DDH oracle checks this. Boneh et al. prove:

Theorem 3.2 ([?]). *Let G be a GDH group. Then $GS[G]$ is a secure signature scheme in the random oracle model.*

What makes BLS attractive for thresholdization is that signing is a single exponentiation with a secret exponent, and verification is a DDH check. Since exponentiation is linear in the exponent, distributing x among n parties and having each compute a partial signature $H(M)^{x_i}$ gives you a natural basis for reconstruction. There are no random nonces involved, so no interaction is needed during signing.

3.3 The Threshold GDH Signature Scheme TGS

3.3.1 Security Model and Definitions

The n players are $P_1, \dots, P_n$. They communicate over a broadcast channel and pairwise secure point-to-point channels. The adversary can corrupt up to t players.

A (t, n) -threshold secret sharing $(s_1, \dots, s_n) \xrightarrow{(t, n)} s$ means any $k \leq t$ shares reveal no information about s , and any $t + 1$ shares reconstruct s efficiently.

A threshold signature scheme $TS = (\mathcal{TK}, \mathcal{TS}, \mathcal{V})$ consists of a distributed key generation algorithm $\mathcal{TK}$, a distributed signing algorithm $\mathcal{TS}$, and a (standard) verification algorithm $\mathcal{V}$.

Definition 3.3 ([2]). *A threshold signature scheme TS is secure robust if:*

1. **Unforgeability**: *No polynomial-time adversary corrupting up to t players can produce a valid pair (M, σ) for a message M not submitted to $\mathcal{TS}$ as public input.*
2. **Robustness**: *For every adversary corrupting up to t players, both $\mathcal{TK}$ and $\mathcal{TS}$ complete successfully.*

3.3.2 Key Generation: No Trusted Dealer

This is perhaps the most important difference from Shoup's scheme. *TGS* does not use a trusted dealer. Instead, $\mathcal{TK}$ is the distributed key generation protocol *DKG* of Gennaro, Jarecki, Krawczyk, and Rabin [?].

DKG is executed jointly by all n players. At the end, each player P_i holds a secret share x_i such that $(x_1, \dots, x_n) \xrightarrow{(t,n)} x$, where $x = \log_g y$ and y is the public key output of the protocol. No single party ever knows x . Furthermore, *DKG* outputs verification information $B_i = g^{x_i}$ for each player, which is publicly available. These B_i values are what enable share verification without zero-knowledge proofs.

The correctness guarantees of *DKG* [?] ensure that in the presence of up to $t < n/2$ corrupted parties, all subsets of $t + 1$ shares define the same unique x , and x is uniformly distributed in $\mathbb{Z}_p^*$.

3.3.3 Signing Protocol

Any subset R of more than t players can sign a message M . Each player $P_i \in R$ computes its signature share:

$$\sigma_i = H(M)^{x_i}$$

and broadcasts σ_i . A designated combiner D (any player or external party) then:

1. Checks validity of each share by verifying $\mathcal{V}_{DDH}(g, B_i, H(M), \sigma_i) = 1$. If a share fails, the corresponding player is treated as malicious and excluded. If fewer than $t + 1$ valid shares remain, signing fails.
2. With a set R of at least $t + 1$ valid shares, reconstructs the signature:

$$\sigma = \prod_{i \in R} \sigma_i^{L_i},$$

where $L_i = \prod_{j \in R, j \neq i} \frac{j}{j-i} \pmod p$ is the Lagrange coefficient for index i over the set R at point 0.

The output is (M, σ) .

Why this works. The Lagrange interpolation identity gives $\sum_{i \in R} L_i x_i = x \pmod p$. Therefore:

$$\prod_{i \in R} \sigma_i^{L_i} = \prod_{i \in R} H(M)^{x_i L_i} = H(M)^{\sum_{i \in R} L_i x_i} = H(M)^x = \sigma.$$

The group order p is public, so L_i are computable over $\mathbb{Z}_p$ directly. This is the critical difference from Shoup: there is no need for $\Delta = l!$ because there is no hidden $\phi(n)$.

Robustness via DDH. The share verification step $\mathcal{V}_{DDH}(g, B_i, H(M), \sigma_i) = 1$ checks that $\sigma_i = H(M)^{x_i}$, i.e., that P_i used its correct secret share. This works because $B_i = g^{x_i}$ is public, so $(g, B_i, H(M), \sigma_i)$ being a DH tuple is exactly the condition that σ_i was computed correctly. The DDH oracle (provided by the pairing) decides this efficiently. No zero-knowledge proofs are needed.

3.4 Security Analysis

Theorem 3.4 (Theorem 3.2, [2]). *Let G be a GDH group. Then $TGS[G]$ is a secure robust threshold signature scheme in the random oracle model against any adversary that corrupts $t < n/2$ players.*

The proof has two parts.

Robustness. Robustness of $\mathcal{TK}$ follows from the correctness guarantees of DKG [?]: with up to $t < n/2$ corrupted parties, DKG completes and produces the right distribution on shares. Robustness of $\mathcal{TS}$ follows from share verification: since D checks $\mathcal{V}_{DDH}(g, B_i, H(M), \sigma_i) = 1$ before using any share, bad shares from corrupted players are detected and discarded. As long as at least $t + 1$ honest players participate (which is guaranteed since $t < n/2$ and $n - t > n/2 > t$), signing succeeds.

Unforgeability. The proof uses a general lemma from [?]: if the underlying base signature scheme S is secure and the threshold scheme TS is *simulatable*, then TS is unforgeable. Simulatability means there exists a simulator SIM that, given only the public key y and an RSA signing oracle (or here, a BLS signing oracle), can produce a view of the $\mathcal{TK}$ and $\mathcal{TS}$ protocols that is computationally indistinguishable from the real view.

The simulator SIM works as follows. For $\mathcal{TK}$, it uses the DKG simulator from [?], which correctly simulates the key generation view for an adversary corrupting players with indices $1, \dots, t'$ (where $t' \leq t$). SIM knows all shares $x_{t'+1}, \dots, x_{n-1}$ and can compute signature shares $\sigma_i = H(M)^{x_i}$ directly for those players. For the last share σ_n , SIM computes it as $\sigma_n = \sigma \cdot \prod_{i \in R} (\sigma_i)^{-1}$ where σ is obtained from the BLS signing oracle. Since by construction σ_n corresponds to the implicitly defined x_n , and DKG 's guarantees ensure all shares have the right distribution, the simulated view is indistinguishable from real.

Given simulatability, the reduction to CDH (via the security of $GS[G]$) is straightforward: any adversary that forges TGS signatures would break the security of the underlying GS . The random oracle is used only in the proof of security of GS , not in the threshold protocol itself.

Threshold $t < n/2$. This bound comes from *DKG*. The protocol requires a strict majority of honest players, i.e., $n - t > t$, giving $t < n/2$. Shoup's Protocol 1 allows up to $t = k - 1$ corruptions with k as low as 2, which can tolerate t up to $n - 2$. The $t < n/2$ bound in *TGS* is more restrictive. However, *TGS* does not need a trusted dealer, which compensates in practice.

3.5 Proactive Security

Proactive security addresses a long-term adversary that compromises fewer than t parties in each time period, but different parties across periods. Over time it might accumulate information from more than t parties total. The solution is to periodically *refresh* the shares, so that knowledge from a previous period is useless after refreshing.

Boldyreva shows that *TGS* can be made proactively secure using the proactive secret sharing protocol *PSS* of Herzberg et al. [?]. The requirements for this are: the threshold key generation must implement Shamir's secret sharing of $x = \log_g y$, the scheme must output verification information $(g^{x_1}, \dots, g^{x_n})$ (so shares can be verified after each refresh), and the signing protocol must be simulatable.

TGS satisfies all three: *DKG* outputs Shamir shares, verification information $B_i = g^{x_i}$ is already part of the public output, and simulatability was established above. Thus, applying *PSS* to *TGS* gives a proactively secure threshold signature scheme with no additional protocol changes.

3.6 The GDH Multisignature Scheme MGS

A multisignature scheme lets a group of signers collectively sign a message and produce a single compact signature. The resulting signature should convince a verifier that every member of the specified group signed, while being no larger than a standard single signature.

The *MGS* scheme is simple. Each player P_j with secret key x_j and public key g^{x_j} independently computes $\sigma_j = H(M)^{x_j}$ and broadcasts it. The combined signature is:

$$\sigma = \prod_{j \in L} \sigma_j = H(M)^{\sum_{j \in L} x_j}.$$

The verifier computes the combined public key $pk_L = \prod_{j \in L} g^{x_j} = g^{\sum_{j \in L} x_j}$ and checks $\mathcal{V}_{DDH}(g, pk_L, H(M), \sigma) = 1$.

Correctness is immediate: the product of the σ_j is an exponentiation of $H(M)$ by the sum of the secret keys, and the product of the public keys is g raised to the same sum.

A notable feature is that the group of signers L does not need to be fixed in advance. This solved an open problem from Micali, Ohta, and Reyzin [?], whose scheme required

all signers to know the full set L before computing their shares. In MGS , each player computes σ_j independently without knowing who else will sign.

Theorem 3.5 (Theorem 4.2, [2]). *Let G be a GDH group. Then $MGS[G]$ is a secure multisignature scheme in the random oracle model.*

The proof reduces multisignature security to the security of $GS[G]$. An adversary B for $GS[G]$ simulates the multisignature game for an adversary A : it gives A the public key $y = g^x$ for the honest player and lets A generate the other $n - 1$ key pairs. Whenever A asks the honest player to participate in signing, B forwards the query to its own signing oracle. When A outputs a forged multisignature on M , B extracts a forgery on M for the underlying GS scheme.

3.7 The Blind GDH Signature Scheme BGS

A blind signature scheme lets a user obtain a signature from a signer without the signer learning which message was signed. The user “blinds” the message, gets a signature on the blinded version, and then “unblinds” to recover a valid signature on the original message. The signer should not be able to link the resulting signature to any particular signing session.

The BGS scheme works as follows. The user holds a public key (p, g, H, y) for the signer (where $y = g^x$). To get a signature on M :

1. User picks random $r \leftarrow \mathbb{Z}_p^*$, computes $\bar{M} = H(M) \cdot g^r$, and sends $\bar{M}$ to the signer.
2. Signer computes $\bar{\sigma} = \bar{M}^x$ and returns it.
3. User computes $\sigma = \bar{\sigma} \cdot y^{-r}$.

Correctness: $\sigma = \bar{M}^x \cdot y^{-r} = (H(M) \cdot g^r)^x \cdot g^{-rx} = H(M)^x \cdot g^{rx} \cdot g^{-rx} = H(M)^x$, which is a valid GS signature on M .

Blindness holds because the signer sees only $\bar{M} = H(M) \cdot g^r$. Since r is uniformly random in $\mathbb{Z}_p^*$, g^r is a uniform random element of G , so $\bar{M}$ is independent of $H(M)$. The signer learns nothing about M .

For unforgeability, the usual EUF-CMA notion does not apply to blind signatures: by design, the user can produce a valid signature on any message after one signing session. The right notion is security against *one-more-forgery*: after l signing interactions, the user cannot produce $l + 1$ valid signatures.

Definition 3.6 (Chosen-target CDH, [2]). *Given (p, g, H, y) , a target oracle returning random group elements z_i , and a helper oracle computing $(\cdot)^x$, the adversary’s goal is to output l pairs (v_i, z_{j_i}) with $v_i = z_{j_i}^x$, using fewer than l queries to the helper oracle. The chosen-target CDH assumption states no polynomial-time adversary can do this.*

This assumption is analogous to the chosen-target RSA inversion assumption used to prove security of Chaum’s RSA blind signature [?]. With one query to the target oracle, it reduces to standard CDH, so it is a natural extension.

Theorem 3.7 (Theorem 5.3, [2]). *If the chosen-target CDH assumption holds in G , then $BGS[G]$ is secure against one-more-forgery under chosen message attack.*

The proof builds a reduction B that simulates BGS for the adversary A . B forwards each hash query from A to the target oracle (getting a random z_i in G), and forwards each blind signing query to the helper oracle (computing z_i^x). When A outputs l signed pairs, B matches them to its list of target oracle outputs and returns the corresponding pairs, having made fewer than l helper queries overall.

3.8 Observations and Critique

The TGS scheme is a significant improvement over RSA-based threshold schemes in terms of simplicity. The signing protocol is essentially the same as BLS signing, with Lagrange interpolation as the only addition. There are no zero-knowledge proofs, no Δ scaling, and no safe prime requirements.

The absence of a trusted dealer is a genuine practical advantage. In Shoup’s scheme, the dealer knows the full private key at setup, which is a security risk. In TGS , the DKG protocol distributes key generation so that no single party ever knows x . This matters in settings like blockchain nodes or threshold certificate authorities where a trusted third party for setup is not available.

The $t < n/2$ corruption bound is a limitation. Shoup’s scheme, with a trusted dealer, can tolerate up to $t = n - 2$ corruptions (any $k \geq 2$ threshold). Boldyreva’s scheme is more restrictive. This is an inherent cost of using DKG for dealer-free key generation rather than a flaw in the threshold signing protocol itself. If a trusted dealer is acceptable, the threshold could be improved, but that would lose the dealer-free property that makes this scheme attractive.

The paper also does not provide a fully detailed proof of Theorem 3.2 in the main body; the proof is in Appendix A and relies heavily on the machinery of [?] and [?]. For a reader who wants to fully verify the security argument, working through those two papers is necessary. The reduction is modular and clean, but it is not self-contained.

Compared to prior threshold DSS schemes [?], TGS requires far less interaction during signing (one round of broadcasting shares vs. multiple rounds in DSS) and avoids error-correction techniques. The trade-off is that TGS requires GDH groups (i.e., pairing-friendly curves), while threshold DSS works in any DL group. In practice, pairing-friendly curves are widely available and used, so this is not a significant restriction.

4 FROST: Flexible Round-Optimized Schnorr Threshold Signatures

4.1 Introduction and Motivation

Threshold signatures enable a group of n participants to collectively produce a signature, requiring at least t (the threshold) participants to cooperate. This primitive is essential for decentralised systems such as cryptocurrency wallets, certificate authorities, and blockchain consensus. Among signature schemes, Schnorr signatures are attractive for thresholdisation due to their linearity and provable security under the discrete logarithm (DL) assumption. However, prior threshold Schnorr schemes (Gennaro et al., 2003; Stinson and Stroh, 2001) required at least three communication rounds per signing operation, each involving broadcasts from all participants—a prohibitive overhead for network-limited or asynchronous environments.

FROST (Flexible Round-Optimized Schnorr Threshold signatures), introduced by Komlo and Goldberg (2021), prioritises round efficiency over robustness. It achieves signing in two rounds (interactive) or a single round (with preprocessing) by replacing distributed key generation for each nonce with non-interactive additive secret sharing and share conversion (Cramer, Damgård, Ishai, 2005). Standardised as RFC 9591 (2024) for curves Ed25519, Ed448, P-256, Ristretto255, and secp256k1, FROST has seen adoption in Zcash and other platforms.

This review provides a self-contained exposition of FROST: background, mathematical assumptions, complete proofs of correctness and security, efficiency analysis, an overview of the ROAST extension, and a critical conclusion.

4.2 Background and Prerequisites

4.2.1 Notation

Let $\mathbb{G}$ be a cyclic group of prime order q with generator g . The DL problem: given g and g^x , compute x . Let $H : \{0, 1\}^* \rightarrow \mathbb{Z}_q^*$ be a random oracle. Participants P_i have identifiers $i \in \{1, \dots, n\}$. For a set S of t identifiers, the Lagrange coefficient is $\lambda_{i,S} = \prod_{j \in S, j \neq i} \frac{j}{j-i} \bmod q$.

4.2.2 Shamir Secret Sharing

To share a secret $s \in \mathbb{Z}_q$ with threshold t , a dealer picks a random degree $t-1$ polynomial $f(x) = \sum_{k=0}^{t-1} a_k x^k$ with $f(0) = s$. Each P_i gets share $s_i = f(i)$. Any t parties reconstruct $s = \sum_{i \in S} \lambda_{i,S} s_i$.

4.2.3 Additive Secret Sharing and Share Conversion

In a t -out-of- t additive scheme, each P_i chooses $s_i \in_R \mathbb{Z}_q$ and the secret is $s = \sum s_i$ (no interaction). Cramer, Damgård, Ishai (2005) give a non-interactive conversion to Shamir shares for $n = t$: define $g_{\{i\}}(x) = \prod_{j \neq i} \frac{j-x}{j}$. Then $f(x) = \sum s_i g_{\{i\}}(x)$ has degree $t-1$, $f(0) = s$, and $f(i) = s_i / \lambda_{i,S}$. Hence $\lambda_{i,S} f(i) = s_i$ are additive shares. This conversion requires no communication.

4.2.4 Distributed Key Generation (DKG)

Pedersen's DKG (Pedersen, 1991) allows n parties to jointly generate a secret without a trusted dealer. Each participant acts as a dealer in Feldman's VSS (Feldman, 1987). In the first round, each broadcasts commitments to the coefficients of a random polynomial. In the second round, they distribute shares to other participants and verify consistency. The final share of P_i is the sum of received shares; the group public key is the product of all constant-term commitments. Gennaro et al. (2007) proved security under honest majority assumptions.

4.2.5 Schnorr Signatures

A Schnorr signature on m with secret key s and public key $Y = g^s$ is: sample $k \in_R \mathbb{Z}_q$, compute $R = g^k$, $c = H(m, R)$, $z = k + s \cdot c \bmod q$, output $\sigma = (z, c)$. Verification checks $c \stackrel{?}{=} H(m, g^z Y^{-c})$.

4.3 The FROST Threshold Signature Scheme

4.3.1 Assumptions

FROST assumes: (i) participants validate messages before signing; (ii) reliable message delivery; (iii) authenticated participant identifiers. A semi-trusted *signature aggregator* $\mathcal{A}$ coordinates signing but cannot forge or learn keys; malicious $\mathcal{A}$ can only cause denial of service.

4.3.2 Key Generation (Ped-DKG)

The key generation protocol proceeds in two rounds. In the first round, each participant P_i picks a random degree $t-1$ polynomial $f_i(x) = \sum_{j=0}^{t-1} a_{ij} x^j$ and broadcasts a commitment vector $\vec{X}_i = \langle g^{a_{i0}}, \dots, g^{a_{i(t-1)}} \rangle$ to all other participants. In the second round, each P_i securely sends the share $f_i(j)$ to each other participant P_j . Upon receiving a share from P_j , participant P_i verifies it using the public commitments:

$$g^{f_j(i)} \stackrel{?}{=} \prod_{k=0}^{t-1} \phi_{jk}^{i^k \bmod q},$$

where $\phi_{jk} = g^{a_{jk}}$. If verification fails, the protocol aborts and the misbehaving participant is investigated out-of-band. Otherwise, each participant computes their long-lived secret share as $s_i = \sum_{j=1}^n f_j(i)$. The group public key is $Y = \prod_{j=1}^n g^{a_{j0}}$, and each participant's public verification share is $Y_i = g^{s_i}$.

4.3.3 Two-Round Interactive Signing

The signing protocol uses two rounds. Let S be the set of t participants selected for signing. To prevent concurrent-session attacks, each participant generates two nonces: a hiding nonce d_i and a binding nonce e_i .

In the first round, the signature aggregator $\mathcal{A}$ requests commitment shares from each participant in S . Each P_i samples fresh nonces $d_i, e_i \in_R \mathbb{Z}_q$, computes the corresponding public commitment shares $D_i = g^{d_i}$ and $E_i = g^{e_i}$, returns (D_i, E_i) to $\mathcal{A}$, and stores (d_i, e_i, D_i, E_i) locally.

In the second round, $\mathcal{A}$ computes the group commitments $R = \prod_{i \in S} D_i$ and $R' = \prod_{i \in S} E_i$. It then sends the tuple (m, R, R', S) to each P_i . Each participant validates the message m ; if invalid, they abort. They compute the challenge $c = H(m, R)$ and the binding factor $\rho_i = H(m, R, R', i)$. Then they compute their response share as

$$z_i = d_i + e_i \cdot \rho_i + c \cdot \lambda_{i,S} \cdot s_i \text{ mod } q.$$

After computing z_i , they securely delete (d_i, e_i) and return z_i to $\mathcal{A}$.

The aggregator verifies each response share by checking

$$g^{z_i} \stackrel{?}{=} D_i \cdot E_i^{\rho_i} \cdot Y_i^{c\lambda_{i,S}}.$$

If any check fails, the aggregator identifies and reports the misbehaving participant and aborts. If all checks pass, $\mathcal{A}$ computes the group response $z = \sum_{i \in S} z_i$ and outputs the signature $\sigma = (z, c)$.

Correctness: The hiding nonces d_i are additive shares of $k_{\text{hide}} = \sum d_i$, and the binding nonces e_i contribute $\sum e_i \rho_i$. The effective nonce is $k = k_{\text{hide}} + \sum e_i \rho_i$. By share conversion, $\lambda_{i,S} s_i$ are additive shares of the long-term secret s . Then $z = \sum d_i + \sum e_i \rho_i + c \sum \lambda_{i,S} s_i = k + sc$, matching the single-party Schnorr signing equation.

4.3.4 Single-Round Signing with Preprocessing

To achieve a single round of interaction, FROST introduces an optional non-interactive batched preprocessing stage. In the preprocessing stage, each participant P_i generates ζ pairs of nonce-commitment tuples $(d_{ij}, e_{ij}, D_{ij}, E_{ij})$ for $j = 1, \dots, \zeta$, where $d_{ij}, e_{ij} \in_R \mathbb{Z}_q$, $D_{ij} = g^{d_{ij}}$, $E_{ij} = g^{e_{ij}}$. The participant stores these tuples locally and publishes the list of commitment shares $\{(D_{ij}, E_{ij})\}$ (along with their identifier i) to a commitment

server. The commitment server stores these values for future signing operations, deleting or marking as used each commitment once it is served.

When a signature on message m is required, the aggregator $\mathcal{A}$ fetches the next unused commitment share pair (D_{ij}, E_{ij}) for each participant in the selected set S from the commitment server. The server returns these values and removes them to prevent reuse. $\mathcal{A}$ computes the group commitments $R = \prod_{i \in S} D_{ij}$ and $R' = \prod_{i \in S} E_{ij}$, then sends $(m, R, R', S, D_{ij}, E_{ij})$ to each P_i . Each participant verifies that D_{ij} and E_{ij} correspond to valid unused nonces (i.e., that they have not been deleted or used before). If verification fails, the participant aborts and requests a restart. Otherwise, the protocol proceeds exactly as in the second round of the two-round variant: each participant computes $c = H(m, R)$, $\rho_i = H(m, R, R', i)$, then $z_i = d_{ij} + e_{ij}\rho_i + \lambda_{i,S}s_i c$, deletes the nonce pair, and returns z_i to $\mathcal{A}$. The aggregator verifies and aggregates as before. Because the first round is eliminated, signing becomes a single round of communication, and participants can respond asynchronously.

4.4 Mathematical Proofs of Security

4.4.1 Assumptions

Definition 4.1 (Discrete Logarithm Problem (DL)). *Given g and $h = g^x$ for unknown $x \in_R \mathbb{Z}_q$, no probabilistic polynomial-time algorithm outputs x with non-negligible probability.*

Definition 4.2 (One-More Discrete Logarithm (OMDL)). *An adversary with access to a challenge oracle (providing random group elements) and a solution oracle (returning discrete logarithms) cannot solve $\ell+1$ challenges with only ℓ queries to the solution oracle.*

Adaptive security proofs may also use the Algebraic Group Model (AGM), where adversaries must represent each group element as a linear combination of previously seen elements.

4.4.2 Correctness Proof

Theorem 4.3 (Correctness). *If all participants follow the FROST signing protocol honestly, the resulting signature $\sigma = (z, c)$ satisfies the Schnorr verification equation.*

Proof. Let $f_1(x)$ be the polynomial defined during key generation such that $f_1(0) = s$ (the group secret) and $f_1(i) = s_i$ (the long-lived secret share of P_i). Let $f_2(x)$ be the polynomial constructed from the additive hiding nonce shares via share conversion, satisfying $f_2(0) = \sum d_i$ and $f_2(i) = d_i/\lambda_{i,S}$. Similarly, let $f_3(x)$ be the polynomial from binding nonce shares with $f_3(0) = \sum e_i$ and $f_3(i) = e_i/\lambda_{i,S}$.

Define the effective nonce $k = f_2(0) + \sum_{i \in S} \lambda_{i,S} f_3(i) \rho_i$. The binding factors ρ_i are publicly derived from the message and all commitments. Each participant computes $z_i = d_i + e_i \rho_i + \lambda_{i,S} s_i c = \lambda_{i,S} (f_2(i) + f_3(i) \rho_i + c f_1(i))$. The aggregator computes $z = \sum_{i \in S} z_i = \sum_{i \in S} \lambda_{i,S} (f_2(i) + f_3(i) \rho_i + c f_1(i))$. By Lagrange interpolation, $\sum \lambda_{i,S} f_2(i) = f_2(0)$, $\sum \lambda_{i,S} f_3(i) \rho_i = \sum e_i \rho_i$ (since ρ_i are public constants), and $\sum \lambda_{i,S} f_1(i) = f_1(0) = s$. Hence $z = f_2(0) + \sum e_i \rho_i + cs = k + sc$. Verification then gives:

$$g^z \cdot Y^{-c} = g^{k+sc} \cdot g^{-sc} = g^k = \left(\prod_{i \in S} D_i \right) \cdot \left(\prod_{i \in S} E_i^{\rho_i} \right) = R \cdot \prod E_i^{\rho_i},$$

which is consistent with the challenge computation because $c = H(m, R)$ and the binding factors are derived from m, R, R' . Thus the signature verifies correctly. $\square$

4.4.3 Unforgeability (EUF-CMA)

Theorem 4.4 (EUF-CMA Security). *If the discrete logarithm problem is hard in $\mathbb{G}$, then FROST is existentially unforgeable under adaptive chosen-message attacks when the adversary controls fewer than t participants.*

Proof Sketch. The proof reduces the security of FROST to that of single-party Schnorr signatures. Let $\mathcal{A}_{\text{Thresh}}$ be an adversary against FROST that controls at most $t - 1$ participants. We construct an adversary $\mathcal{A}_{\text{Schnorr}}$ against the single-party Schnorr scheme.

$\mathcal{A}_{\text{Schnorr}}$ receives a challenge public key Y^* and simulates the entire FROST environment for $\mathcal{A}_{\text{Thresh}}$. For key generation, $\mathcal{A}_{\text{Schnorr}}$ sets the group public key to Y^* . For the $t - 1$ corrupted participants, it chooses random secret shares. For the single honest participant, it implicitly defines the share as the difference between Y^* and the contributions of corrupted parties, using the homomorphic property of Pedersen commitments.

When $\mathcal{A}_{\text{Thresh}}$ requests a signature on a message m , $\mathcal{A}_{\text{Schnorr}}$ queries its own signing oracle to obtain a single-party Schnorr signature (z, c) on m . It then simulates the FROST signing protocol: it generates random nonce shares (d_i, e_i) for the corrupted participants, computes the group commitments R, R' from these simulated shares, and ensures consistency with the challenge c by programming the random oracle H appropriately (setting $H(m, R) = c$ and defining binding factors ρ_i accordingly).

If $\mathcal{A}_{\text{Thresh}}$ outputs a valid FROST forgery (z^*, c^*) on a new message m^* (one never signed before), the reduction extracts a valid Schnorr forgery by reconstructing the honest participant's contribution. Because the adversary controls fewer than t parties, at least one honest participant contributed to the forgery, and the extraction succeeds. The probability of success of $\mathcal{A}_{\text{Schnorr}}$ is polynomially related to that of $\mathcal{A}_{\text{Thresh}}$. Since the DL assumption implies that single-party Schnorr is unforgeable, the advantage of $\mathcal{A}_{\text{Thresh}}$ must be negligible. The full proof requires careful handling of the random oracle and the algebraic properties of share conversion. $\square$

4.4.4 Adaptive Security Considerations

The original FROST security proof assumes a static corruption model, where the adversary chooses which participants to corrupt before the protocol begins. Crites and Stewart (2025) demonstrated that this proof does not extend to adaptive adversaries, where corruption can occur after seeing protocol messages. Later, Crites, Katz, Komlo, Tessaro, and Zhu (2025) proved adaptive security for a variant of FROST under the algebraic one-more discrete logarithm (A-OMDL) assumption combined with the low-dimensional vector representation (LDVR) problem, all within the algebraic group model. These assumptions remain non-standard, and achieving adaptive security under standard DL assumptions remains an open problem.

4.5 Security and Efficiency Analysis

4.5.1 Security Properties

FROST provides unforgeability (under the DL assumption), privacy of individual secret shares, and identifiable aborts. It explicitly trades away *robustness*: if any participant misbehaves (e.g., sends an incorrect response share), the protocol aborts rather than attempting to continue with the remaining honest participants. This design is justified for use cases where misbehaviour is rare and can be investigated out-of-band, such as single-owner device groups or environments where abort triggers immediate investigation.

An important nuance regarding security levels was identified by Bellare, Tessaro, and Zhu (2022). They defined a hierarchy of unforgeability notions for threshold signatures, ranging from TS-SUF-0 (weakest) to TS-SUF-4 (strongest). The original FROST specification (often called FROST1) achieves TS-SUF-3, which is a strong level of unforgeability. However, a later optimized variant (FROST2), introduced to reduce the number of exponentiations required per signer, inadvertently degrades to TS-SUF-2, providing weaker security guarantees. Implementers should be aware that not all FROST variants offer the same security level; the RFC 9591 standard follows the original FROST1 design.

4.5.2 Efficiency Comparison

The following table compares FROST with prior threshold Schnorr schemes in terms of communication rounds, message complexity, and robustness.

Scheme	Signing Rounds	Message Complexity	Robust
Gennaro et al. (2003)	3	$O(n^2)$ per round	Yes
Stinson-Strobl (2001)	≥ 4	$O(n^2)$ per round	Yes
FROST (2-round)	2	$O(t)$ (aggregator)	No
FROST (1-round w/ prep)	1	$O(t)$ (aggregator)	No

For a signing operation involving t participants, FROST requires:

- **Communication:** The aggregator sends $O(t)$ messages in each round and receives $O(t)$ responses. This contrasts with broadcast-based schemes where each participant sends $O(n)$ messages per round, totalling $O(n^2)$.
- **Computation:** Each participant performs a constant number of exponentiations per signing (two for D_i, E_i , one for the response share, and verification of incoming shares). The aggregator performs $O(t)$ exponentiations for verification.
- **Preprocessing:** The single-round variant requires offline generation of nonce-commitment pairs. This trades storage and preprocessing latency for lower online latency.

4.5.3 Drawbacks and Limitations

FROST exhibits several limitations identified in subsequent literature:

1. **Preprocessing Dependence:** The single-round variant requires a reliable commitment server. If the server fails or serves stale commitments, the protocol may abort or cause nonce reuse.
2. **Lack of Proactive Security:** FROST does not support periodic refresh of secret shares. In long-term deployments, gradual compromise of shares cannot be remedied without re-running key generation.
3. **Static Participants:** The protocol assumes a fixed set of participants after key generation. Dynamic joins or removals are not supported.
4. **No Public Accountability:** Misbehaviour only causes an abort; there is no mechanism to publicly blame a malicious participant without revealing secrets.
5. **Static Corruption Model:** The original security proof assumes a fixed set of corrupted participants before the protocol starts, which is insufficient for adaptive adversaries.
6. **No Post-Quantum Security:** FROST relies on the discrete logarithm problem, which is vulnerable to quantum attacks (Shor’s algorithm).

4.6 Overview of ROAST

ROAST (Robust Asynchronous Schnorr Threshold Signatures), introduced by Ruffing, Ronge, Jin, Schneider-Bensch, and Schröder at CCS 2022, is an extension that addresses

FROST’s lack of robustness. ROAST operates as a wrapper around FROST, enabling signing to continue despite misbehaving participants.

ROAST adds the following features to FROST:

- **Asynchronous Operation:** Multiple concurrent signing sessions are supported. Participants can be slow or temporarily offline without blocking the protocol.
- **Robustness via Redundancy:** By generating a sufficient number of preprocessed nonce-commitment pairs, ROAST can discard contributions from misbehaving participants and still complete the signature using any t honest participants.
- **Identifiable Aborts with Continuation:** Malicious participants are identified and excluded, but the protocol does not abort; it continues with the remaining honest participants.

The ROAST protocol modifies the preprocessing stage to produce an excess of nonce pairs. When a participant sends an invalid response share, the aggregator simply requests a fresh nonce from that participant (or from a replacement) using a different preprocessed commitment. Because preprocessing is batched, this can be done without starting a new signing round.

Trade-offs of ROAST include:

- Higher computational overhead from managing concurrent sessions and redundant nonces.
- Increased communication complexity due to additional messages for requesting replacement nonces.
- Weaker unforgeability guarantees: Bellare, Tessaro, and Zhu (2022) showed that while original FROST1 achieves TS-SUF-3, ROAST only achieves TS-SUF-2.

ROAST is particularly suitable for asynchronous environments where participants may have unreliable connectivity, such as decentralised networks, blockchain validators, and IoT systems with intermittent connectivity.

4.7 Conclusion and Future Research Directions

FROST advances threshold Schnorr signatures by reducing signing rounds to two (or one with preprocessing) via non-interactive additive secret sharing and share conversion. Its standardisation as RFC 9591 enables practical deployment in systems like Zcash. However, its lack of robustness, adaptive security (recently addressed only under non-standard assumptions), proactive refresh, and dynamic participant management limits long-term use. Open problems include: constructing adaptively secure FROST under

standard DL assumptions; post-quantum lattice or hash-based variants; efficient dynamic threshold and proactive secret sharing; accountable robustness with public blame; formal verification; and standardisation of robust extensions like ROAST. Addressing these will make FROST suitable for a broader range of critical applications.

References

- [1] V. Shoup, “Practical threshold signatures,” *Advances in Cryptology – Eurocrypt 2000*, LNCS 1807, pp. 207–220, Springer, 2000.
- [2] A. Boldyreva, “Threshold signatures, multisignatures and blind signatures based on the Gap-Diffie-Hellman-group signature scheme,” *Public Key Cryptography – PKC 2003*, LNCS 2567, pp. 31–46, Springer, 2003.
- [3] C. Komlo and I. Goldberg, “FROST: Flexible Round-Optimized Schnorr Threshold Signatures,” *Selected Areas in Cryptography – SAC 2020*, LNCS 12804, pp. 34–65, Springer, 2021.